

[+ Code](#)[+ Text](#)

Curso de Nivelamento: Python para Robótica

Módulo 8: Programação Orientada a Objetos (POO)

Objetivo

Entender os princípios da Programação Orientada a Objetos (POO) em Python e como aplicá-los no contexto da robótica, preparando o estudante para estruturas comuns em sistemas ROS 2, onde nós são frequentemente implementados como classes.

1. O que é POO?

A Programação Orientada a Objetos é uma forma de organizar o código baseada em **objetos**, que representam entidades com **características (atributos)** e **ações (métodos)**.

No contexto da robótica:

- Um robô pode ser representado como um objeto
 - Seus sensores e atuadores podem ser atributos
 - Suas ações (andar, medir, parar) podem ser métodos
-

2. Atributos e Métodos

- **Atributos** são variáveis associadas a um objeto (ex: nome, bateria, temperatura)
- **Métodos** são funções que pertencem ao objeto (ex: mover, verificar_bateria)

Tipos de atributos:

- **Públicos:** podem ser acessados de fora da classe (ex: `self.nome`)
 - **Privados:** são protegidos por convenção usando `_` ou `__` (ex: `self._nivel`, `self.__senha`)
 - Apesar de acessíveis, devem ser tratados como internos da classe
-

3. Criando uma classe

```
class Robo:  
    def __init__(self, nome, bateria):
```

```
self.nome = nome          # atributo público
self._temperatura = 25.0  # atributo protegido (por convenção)
self.bateria = bateria
```

```
def status(self):
    print(f"{self.nome} tem {self.bateria}% de bateria.")
```

- `__init__` é o método construtor, chamado automaticamente ao instanciar
- `self` representa o próprio objeto dentro da classe

4. Criando e usando objetos

```
robo1 = Robo("Neo", 90)
robo1.status()
```

Saída:

```
Neo tem 90% de bateria.
```

5. Adicionando métodos personalizados

```
class Robo:
    def __init__(self, nome, bateria):
        self.nome = nome
        self.bateria = bateria

    def mover(self):
        if self.bateria > 0:
            self.bateria -= 10
```

```
        print(f"{self.nome} se moveu. Bateria: {self.bateria}%")
    else:
        print("Bateria esgotada!")
```

6. Múltiplos objetos e organização

```
r1 = Robo("Alpha", 50)
r2 = Robo("Beta", 20)

r1.mover()
r2.mover()
r2.mover()
```

7. Bloco `if __name__ == "__main__"`

Este bloco é usado para garantir que certas instruções **só sejam executadas se o arquivo for executado diretamente** (e não importado como módulo):

```
if __name__ == "__main__":
    meu_robo = Robo("Zeta", 60)
    meu_robo.status()
```

No ROS 2, scripts principais sempre usam esse bloco para iniciar o nó do robô, evitando que ele seja executado automaticamente ao importar.

8. Organização em arquivos separados

Conforme os projetos crescem, é comum salvar classes em arquivos separados e reutilizá-las com importações.

Exemplo:

Arquivo robo.py:

```
class Robo:
    def __init__(self, nome):
        self.nome = nome
```

Arquivo main.py:

```
from robo import Robo

if __name__ == "__main__":
    r = Robo("Zeta")
    print(r.nome)
```

Isso é especialmente importante em projetos ROS 2, onde diferentes arquivos trabalham juntos.

9. Tarefa Proposta

Crie um script chamado `controle_robo.py` com:

1. Uma classe `Robo` com os atributos `nome`, `bateria` e `temperatura`.
 2. Métodos:
 - `status()`: mostra os dados do robô
 - `verificar_temperatura()`: avisa se a temperatura for maior que 60°C
 3. Instancie dois robôs com dados diferentes e chame seus métodos **dentro do bloco** `if __name__ == "__main__"`.
 4. Como desafio extra, separe a classe em um arquivo chamado `robo.py` e importe no `controle_robo.py`
-

10. Quiz Rápido

1. O que é uma classe em Python?
2. Qual a função do método `__init__()`?
3. O que representa `self` dentro da classe?
4. Qual a utilidade do bloco `if __name__ == "__main__":`?
5. Qual a diferença entre atributo público e protegido?
6. Como importar uma classe salva em outro arquivo?

Respostas:

1. Uma estrutura que define os atributos e comportamentos de um objeto
2. Inicializa os atributos ao criar o objeto
3. Representa o próprio objeto atual
4. Garante que partes do código sejam executadas apenas se o script for chamado diretamente
5. Públicos podem ser acessados diretamente, protegidos devem ser usados apenas dentro da classe (por convenção)
6. Usando `from nome_do_arquivo import NomeDaClasse`

11. Resumo do Módulo

- POO organiza o código em classes, objetos, métodos e atributos
- `self` representa o próprio objeto, e `__init__` define seus atributos
- Atributos podem ser públicos ou protegidos
- O bloco `if __name__ == "__main__":` evita execução indesejada ao importar
- Organizar código em arquivos separados e importar classes é prática comum em projetos maiores, incluindo ROS 2

Parabéns! Você concluiu o curso de nivelamento em Python para Robótica.

A partir daqui, você está preparado para entender scripts de robôs e começar a programar nós no ROS 2.

MetaBee | Formação em Robótica e Tecnologia